

CS 466 Project Report

Title: Deepfake Video Detection Using Frame-Level Neural Networks

Author: Ceren Öz

Abstract

Deepfake videos pose a significant threat to digital media authenticity, making reliable automated detection increasingly important. This project addresses deepfake detection as a frame-based binary classification problem using publicly available datasets, FaceForensics++ for training and Celeb-DF v2 for cross-dataset evaluation. A pretrained EfficientNetV2-B0 model is used as a spatial feature extractor to capture manipulation artifacts from individual facial frames. Video-level predictions are obtained through lightweight aggregation strategies, including probability averaging, majority voting, and feature-level late fusion across frames. To ensure scalability, a full preprocessing pipeline is implemented involving frame extraction, face detection using MTCNN, and efficient storage via LMDB. Experimental results show strong in-domain performance on FaceForensics++ , while performance drops significantly on Celeb-DF v2, indicating limited cross-dataset generalization. The results highlight that while spatial CNN-based features are effective for in-domain detection, they remain sensitive to dataset bias and manipulation style shifts.

1. Introduction

The rapid advancement of deep learning-based face manipulation techniques has led to the emergence of highly realistic deepfake videos. These synthetic videos can convincingly alter identity and facial expressions, raising serious concerns in areas such as misinformation, identity fraud, and media integrity. As a result, developing robust and scalable deepfake detection systems has become an active research problem in computer vision.

This project focuses on detecting deepfake videos using a frame-based learning approach combined with video-level aggregation. Instead of directly modelling temporal dependencies, the system extracts spatial features from individual facial frames using a pretrained EfficientNetV2 backbone and aggregates them to produce final video predictions. This design prioritizes computational efficiency while maintaining strong discriminative capability.

Experiments conducted on FaceForensics++ [3] demonstrate strong in-domain performance, achieving high accuracy and ROC-AUC scores. However, evaluation on Celeb-DF v2 [4] reveals a significant drop in performance, highlighting the challenge of generalization across different deepfake generation methods and datasets.

2. Related Work

2.1. 3D CNNs

3D Convolutional Neural Networks (e.g., C3D) extend standard CNNs by applying convolutions over both spatial and temporal dimensions, allowing direct modelling of motion patterns across frames. These methods are effective in capturing temporal inconsistencies in videos, which are useful for detecting deepfakes. However, 3D CNNs are computationally expensive and require large-scale video datasets to avoid overfitting. In this project, such approaches were not adopted due to limited computational resources

and the relatively small size of training data, making full spatiotemporal learning impractical.

2.2. Two Stream Networks

Two-stream architectures separate spatial appearance (RGB frames) and temporal motion (optical flow) into two parallel networks. The spatial stream captures visual artifacts, while the temporal stream focuses on motion inconsistencies between frames. Although effective in action recognition tasks, these models rely heavily on accurate optical flow estimation, which adds significant preprocessing cost and complexity. In addition, deepfake datasets often contain subtle motion cues that are not consistently captured by optical flow, limiting their reliability in this domain. For these reasons, this project avoids explicit motion modelling and instead relies on learned spatial features with lightweight aggregation.

2.3. Transformer-Based Video Models

Transformer-based architectures, such as TimeSformer [1] and Video Vision Transformers [2], model long-range dependencies across frames using self-attention mechanisms. These models have shown strong performance in video understanding tasks. However, they require large-scale training data and substantial computational resources due to quadratic attention complexity. In the context of deepfake detection, available datasets are relatively limited and often domain-specific, making transformers prone to overfitting. Therefore, this project opts for a more efficient CNN-based feature extractor combined with simple aggregation strategies to balance performance and practicality.

3. Data

This project uses two publicly available deepfake datasets: FaceForensics++ [3] and Celeb-DF v2 [4]. The primary dataset used for training and initial evaluation is FaceForensics++, which contains both authentic and manipulated face videos generated using deepfake techniques. Celeb-DF v2, which contains higher-quality and more realistic fake videos, was used to evaluate the generalization capability of the trained model on unseen data.

The datasets and generated files were stored on Google Drive due to their large size and to enable persistent storage during experiments conducted on Google Colab. The project directory was organized into separate folders for raw videos, extracted frames, cropped face images, metadata files, and processed datasets. The main directory structure included folders such as `raw_videos`, `extracted_frames`, `cropped_faces`, and `metadata`. Separate folders were also reserved for Celeb-DF preprocessing outputs.

3.1. FaceForensics++

The FaceForensics++ dataset contains both real and manipulated videos. In this project, a total of 2,000 videos were used, consisting of 1,000 real videos and 1,000 fake videos. The fake videos were generated using deepfake manipulation methods provided within the dataset. The raw videos were organized according to their manipulation category and compression level. Real videos were stored under the `youtube/c23/videos` directory, while manipulated videos were stored under `Deepfakes/c23/videos`. The `c23` compression setting was selected because it provides a balance between visual quality and storage efficiency.

Since the model operates on facial regions rather than full video frames, extensive preprocessing was required before training. The preprocessing pipeline consisted of two major steps: frame extraction and face cropping.

3.1.1. Frame Extraction

In the first step, frames were extracted from each video using OpenCV. Instead of using every frame from the videos, a frame-skipping strategy was applied to reduce redundancy and storage requirements. Specifically, one frame was saved every five frames (`frame_skip = 5`). The extracted frames were stored in the `extracted_frames` directory while preserving the original video organization. This approach significantly reduced the number of nearly identical consecutive frames while still preserving sufficient temporal diversity.

3.1.2. Face Cropping

After frame extraction, facial regions were detected and cropped from the extracted frames using the Multi-task Cascaded Convolutional Neural Network (MTCNN) implementation provided by the `facenet-pytorch` library. During this step, each frame was processed individually, and only the most prominent face was retained (`keep_all=False`).

The cropped face images were resized to 224×224 pixels to match the model input size. Additional sampling constraints were introduced to control dataset size and reduce bias caused by longer videos. Specifically, only every third extracted frame was processed during face cropping (`FRAME_SKIP = 3`), and a maximum of 30 face images per video was stored (`MAX_FRAMES_PER_VIDEO = 30`).

All extracted and cropped face frames were stored as JPEG images (`*.jpg`) to ensure a good balance between storage efficiency and image quality.

The resulting cropped face images were saved in the `cropped_faces` directory.

3.1.3. Preprocessing Challenges and Robust Pipeline Design

During preprocessing, several practical challenges were encountered. The face-cropping stage required GPU acceleration and took a substantial amount of time when executed on Google Colab. Due to Colab GPU session limitations, the original preprocessing pipeline was interrupted before completion when approximately 87% of the real-video frames had been processed.

To address this issue, the preprocessing pipeline was redesigned to be resume-safe. A tracking file named `processed_videos.txt` was introduced to record completed videos, allowing preprocessing to continue from the last successfully processed video instead of restarting from the beginning. Additional safeguards were also implemented to skip already existing cropped images and handle corrupted or unreadable frames gracefully. This modification significantly reduced repeated computation time and improved preprocessing reliability.

3.1.4. Metadata Creation

Following preprocessing, metadata files were generated using Pandas. For each cropped face image, the metadata included the frame path, corresponding video identifier, and binary class label. Real videos were assigned label 0, while fake videos were assigned label 1. The complete metadata was saved as `frame_metadata.csv`.

To ensure that frames from the same video did not appear in both training and testing sets, the dataset split was performed at the video level rather than the frame level. The unique video identifiers were divided into training and testing subsets using an 80/20 split with Scikit-learn's `train_test_split` function, with shuffling enabled and a fixed random seed (`random_state=42`) to ensure reproducibility.

After preprocessing and filtering, the final dataset contained 25,696 training frames and 6,362 testing frames, corresponding to 937 training videos and 235 testing videos. The resulting metadata for these subsets was stored as `train.csv` and `test.csv`.

3.1.5. Class Imbalance and Dataset Characteristics

An important observation during preprocessing was the emergence of class imbalance. Although the raw dataset initially contained equal numbers of real and fake videos, the final number of extracted face images differed significantly between the classes. This imbalance occurred because face detection performance varied across videos and manipulation types, causing some fake frames to fail during face extraction.

The final processed dataset contained 27,345 real face images and 4,713 fake face images. To reduce the impact of this imbalance during model training, class-weighted loss functions were used in the training stage.

3.1.6. LMDB Storage Optimization

After preprocessing and generating the train/test metadata files, the dataset was further optimized using the LMDB (Lightning Memory-Mapped Database) format to improve data loading efficiency during model training. This step was necessary because the dataset consists of tens of thousands of high-resolution image frames and loading them directly from Google Drive storage would introduce significant I/O bottlenecks during training, particularly under the assumption that multiple models would be trained for experimentation.

In this project, separate LMDB databases were created for the training and testing splits using the previously generated `train.csv` and `test.csv` metadata files. Each sample is stored as a key-value pair where the key is a unique identifier combining video ID and frame name, and the value is a serialized dictionary containing the encoded image, label, video ID, and frame name.

The LMDB creation process was computationally intensive, taking approximately 4 hours and 48 minutes for the training set (25,696 frames) and about 1 hour and 10 minutes for the testing set (6,362 frames). The final databases (`train.lmdb` and `test.lmdb`) were saved to Google Drive for later use.

3.2. Celeb-DF v2

To evaluate cross-dataset generalization, Celeb-DF v2 is used as an unseen test distribution. Unlike FaceForensics++, it is not used during training and therefore provides a strict evaluation setting for robustness under domain shift.

The dataset is obtained using the Kaggle API and contains real celebrity videos (Celeb-real) and synthetic deepfake videos (Celeb-synthesis). The Celeb-synthesis subset includes multiple manipulated versions of the original Celeb-real videos, where each real identity may have several corresponding altered video variants. For this study, a balanced subset of 300 real and 300 fake videos is randomly selected using a fixed random seed to ensure reproducibility.

3.2.1. Frame Extraction

Frames are extracted from each selected video using OpenCV. A frame-skipping strategy (`frame_skip = 5`) is applied to reduce redundancy and computational cost. Extracted frames are stored in structured directories under separate real and fake folders, preserving the mapping between video identity and labels.

Each extracted frame is saved as a JPEG image (*.jpg) to maintain consistency with the FaceForensics++ preprocessing pipeline.

3.2.2. Face Cropping

Facial regions are detected using the MTCNN model from the facenet-pytorch library. Each frame is processed individually, and only the most prominent face is retained (`keep_all=False`). Detected faces are resized to 224×224 pixels to match the model input requirements.

To ensure uniform sampling and control dataset size, only every third extracted frame is processed (`FRAME_SKIP = 3`), and a maximum of 30 face images per video is stored (`MAX_FRAMES_PER_VIDEO = 30`). Frames where no face is detected are automatically discarded.

3.2.3. Metadata Creation

After face cropping, metadata is generated using Pandas. For each cropped face image, the following information is recorded: frame path, video identifier, and binary class label (0 for real, 1 for fake). The resulting metadata is saved as `frame_metadata_celeb_df.csv`.

The final Celeb-DF v2 preprocessing pipeline produces 14,971 cropped face images, which are used for cross-dataset evaluation.

4. Methods

The proposed system formulates deepfake detection as a frame-based visual classification problem, followed by video-level decision aggregation. Instead of learning temporal dynamics directly through sequential models, the approach relies on a strong pretrained 2D CNN backbone to extract spatial artifacts from individual frames and then applies post-hoc fusion strategies to obtain a final video-level prediction.

Two complementary aggregation paradigms are explored within this framework: a baseline frame aggregation approach (post-hoc prediction fusion) and a structured true late fusion model (feature-level aggregation across frames). Both approaches share the same backbone and training philosophy but differ in how multi-frame information is integrated at the video level.

This design choice prioritizes computational efficiency and scalability while maintaining strong performance for deepfake detection, where discriminative cues are often localized in individual frames rather than consistently distributed across time.

4.1. Frame-Level Representation Learning

At the core of both approaches is a 2D convolutional neural network based on EfficientNetV2-B0 (from the `timm` library), pretrained on ImageNet and adapted for binary classification. The final classification head is removed, and the network outputs a feature embedding:

$$\phi(f_t) \in \mathbb{R}^d$$

for each input frame f_t .

EfficientNetV2 is selected due to its favourable trade-off between accuracy and efficiency, as well as its ability to capture fine-grained spatial artifacts such as blending inconsistencies, texture distortions, and compression artifacts that are commonly associated with deepfakes.

All frames are resized to a fixed resolution of 224×224 , normalized, and processed independently without any explicit temporal dependency during feature extraction.

4.2. Dataset Pipeline and LMDB-Based Frame Loading

The dataset is stored in LMDB format, enabling efficient key-value retrieval of compressed frame data. Each stored entry contains a JPEG-encoded frame, a binary label, and a corresponding video identifier.

A custom PyTorch Dataset implementation groups frames by `video_id`, forming:

$$V_i = \{ f_1, f_2, \dots, f_n \}$$

For late fusion approach, a fixed number of frames $T = 8$ is sampled per video. If a video contains more than T frames, random sampling is applied; otherwise, sampling with replacement ensures consistent tensor dimensions.

This strategy enforces a uniform video representation while introducing stochasticity, which acts as a form of regularization across training epochs.

Frames are decoded using OpenCV, converted from BGR to RGB, and augmented using the Albumentations library.

4.3. Data Augmentation Strategy

To improve generalization and robustness to real-world distortions, a set of image-level augmentations is applied during training. These include geometric and photometric transformations such as horizontal flipping, brightness/contrast variation, Gaussian blur, and image compression artifacts.

These augmentations are particularly relevant for deepfake detection, as manipulated videos often exhibit inconsistent compression levels, lighting variations, and subtle blending artifacts. The augmentation pipeline ensures that the model does not overfit to dataset-specific visual patterns.

4.4. Training Configurations

All experiments share a unified training configuration summarized in Table 4.1.

Table 4.1: Shared Training Hyperparameters

Parameter	Value	Description
Image Size	224×224	Input resolution for EfficientNetV2
Batch Size	32	Number of samples per training batch
Epochs	5	Total training iterations over dataset
Learning Rate	$1e - 4$	Step size for optimizer updates
Optimizer	AdamW	Adaptive optimizer with weight decay
Loss Function	BCEWithLogitsLoss	Binary classification loss
Device	CUDA	GPU acceleration
Workers	2	DataLoader parallel loading threads

Training is performed using mixed precision (AMP) with gradient scaling to improve computational efficiency and reduce GPU memory usage. Optimization is carried out using AdamW, which decouples weight decay from gradient updates to improve generalization.

4.5. Baseline Approach: Frame-Level Prediction Aggregation

The baseline system performs independent frame classification, where each frame is passed through the CNN to produce a logit score. Video-level prediction is then computed using post-hoc aggregation of frame-level outputs, without any learned temporal modelling.

Given frame-level probabilities:

$$p_t = \sigma(\phi(f_t))$$

two aggregation strategies are considered:

4.5.1. Average Probability Strategy

The simplest aggregation method computes the mean probability across all frames in a video:

$$p_{video} = \frac{1}{T} \sum_{t=1}^T p_t$$

The final prediction is obtained using a threshold of 0.5:

$$\hat{y} = \begin{cases} 1 & \text{if } p_{video} > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

This method assumes that each frame contributes equally to the final decision and is effective when deepfake artifacts are distributed across multiple frames.

4.5.2. Majority Voting Strategy

In this approach, each frame is first converted into a binary prediction:

$$\hat{y}_t = 1(p_t > 0.5)$$

The final video label is determined by majority voting:

$$\hat{y}_{video} = mode(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_T)$$

This method is more robust to outlier frames but may be sensitive when predictions are highly imbalanced across frames.

4.6. True Late Fusion Approach (Feature-Level Multi-Frame Aggregation)

While the baseline operates on decision-level fusion, the second approach introduces a more structured true late fusion mechanism at the feature level.

Instead of aggregating probabilities, frame-level embeddings extracted from the CNN backbone are aggregated before classification.

4.6.1. Frame Feature Extraction

Each frame is processed independently through EfficientNetV2, producing:

$$\phi(f_t) \in \mathbb{R}^d$$

The resulting tensor is reshaped from:

$$[B, T, d] \rightarrow [B, T, d]$$

where B represents the batch size, T denotes the number of sampled frames per video, and d is the feature dimension produced by the CNN backbone for each frame.

Initially, all frames are merged into a single batch dimension before being passed through the EfficientNetV2 backbone. Therefore, the CNN processes:

$$B \times T$$

individual frames independently, producing a feature matrix of shape:

$$[B, T, d]$$

where each row corresponds to the feature vector of one frame.

After feature extraction, the tensor is reshaped back into:

$$[B, T, d]$$

to restore the temporal grouping structure. In this representation: each batch entry corresponds to one video, each video contains T frame embeddings, and each embedding has dimensionality d .

This reshaping step is necessary before applying mean pooling across the frame dimension.

4.6.2. Feature-Level Aggregation Across Frames

Aggregation is performed using mean pooling across frames:

$$F_{video} = \frac{1}{T} \sum_{t=1}^T \phi(f_t)$$

This produces a single compact representation per video:

$$F_{video} \in \mathbb{R}^{B \times d}$$

This operation assumes that discriminative deepfake artifacts can be captured statistically across multiple frames without requiring explicit sequence modelling or temporal dependency learning.

Compared to recurrent or attention-based models, this method avoids additional temporal parameters, significantly reducing computational complexity.

4.6.3. Video-Level Classification Overhead

The fused representation is passed through a lightweight multi-layer perceptron (MLP):

$$d \rightarrow 512 \rightarrow 128 \rightarrow 1$$

with ReLU activations and dropout regularization (0.3 and 0.2). The final output is a single logit used for binary classification via sigmoid activation.

The classifier learns nonlinear combinations of aggregated spatial features, enabling discrimination between real and manipulated videos.

4.7. Training Objective and Optimization

Both baseline and late fusion models are trained using Binary Cross-Entropy with Logits Loss, defined as:

$$\mathcal{L} = -(y \log \sigma(z) + (1 - y) \log(1 - \sigma(z)))$$

where z represents the model logit.

Optimization is performed using AdamW, with mixed precision training enabled via `torch.cuda.amp`. Gradient scaling ensures numerical stability during backpropagation, particularly under low-precision computation.

4.8. Alternative Approaches and Design Rationale

Alternative architectures such as 3D CNNs, LSTM-based sequence models, and Transformer-based video encoders were considered to explicitly model temporal relationships. However, these approaches introduce significantly higher computational cost and require substantially larger datasets to generalize effectively.

In contrast, the proposed framework deliberately avoids explicit temporal modelling in favour of:

- Strong pretrained spatial feature extraction (EfficientNetV2)
- Lightweight cross-frame aggregation (mean pooling or voting)
- Low computational overhead
- Robustness under limited dataset size

Empirically, this design choice achieves strong performance while maintaining simplicity and scalability, making it suitable as a practical deepfake detection pipeline under constrained computational settings.

5. Experiments

This section evaluates the proposed deepfake detection approaches on FaceForensics++ [3] for in-domain evaluation and Celeb-DF v2 [4] for cross-dataset generalization.

FaceForensics++ is used for training and testing under the same distribution, while Celeb-DF v2 is reserved to assess robustness under unseen, more realistic manipulation styles.

Two experimental settings are considered. The first is a frame-based 2D CNN with post-hoc video-level aggregation, where frame-level predictions are combined after inference. The second is a true late fusion architecture that aggregates multiple frame representations at the feature level prior to classification. The experiments focus on in-domain frame and video-level performance, the effect of different aggregation strategies, and the role of cross-frame feature aggregation in improving video-level decision quality.

Alongside in-domain evaluation, cross-dataset performance on Celeb-DF v2 is used to measure generalization under distribution shift. Training behaviour is additionally tracked using loss, accuracy, and ROC-AUC curves to support analysis of convergence and stability.

5.1. Frame-Based 2D CNN with Post-Hoc Aggregation

The first set of experiments evaluates the baseline frame-based CNN model trained on FaceForensics++. Frame predictions are aggregated after inference to obtain video-level classifications.

5.1.1. FaceForensics++ Results (In-Domain Evaluation)

The model achieves strong frame-level performance on the FaceForensics++ test set, indicating that spatial artifacts extracted from individual frames are sufficient for reliable deepfake detection under in-domain conditions.

Table 5.1: Frame-Level Performance on FaceForensics++ Test Set

Accuracy	Precision	Recall	F1 Score	ROC-AUC
0.9541	0.8844	0.8066	0.8437	0.9832

The high ROC-AUC score shows that the model separates real and fake frames effectively, while the balanced precision and recall values indicate stable classification behaviour.

To obtain video-level predictions, two aggregation strategies are evaluated: Average Probability Aggregation and Majority Voting.

Table 5.2: Video-Level Performance on FaceForensics++ Test Set

Method	Accuracy	Precision	Recall	F1 Score
Average Probability	0.9787	0.9697	0.8889	0.9275
Majority Voting	0.9745	0.9688	0.8611	0.9118

Average Probability Aggregation performs slightly better overall, particularly in recall and F1-score. This suggests that preserving frame-level confidence values provides more informative video-level predictions than converting frame outputs into hard labels before aggregation.

5.1.2. Celeb-DF v2 Results (Cross-Dataset Generalization)

To evaluate robustness under domain shift, the FaceForensics++ model is tested on Celeb-DF v2 without fine-tuning. The evaluation is performed on a balanced subset containing 300 real and 300 fake videos. Video-level predictions are computed using two aggregation strategies: Average Probability Aggregation and Top-K Aggregation (using the highest-confidence frame predictions, in this case top 10%).

Table 5.3: Video-Level Performance on Celeb-DF v2

Method	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Average Probability	0.5000	0.0000	0.0000	0.0000	0.5508
Top-K Aggregation	0.4933	0.3000	0.0100	0.0194	0.5338

The results show a clear degradation in performance under cross-dataset conditions. Both aggregation strategies fail to produce reliable fake detections, with Average Probability collapsing to a trivial “all-real” prediction pattern, resulting in zero recall for the fake class. Although Top-K aggregation slightly increases recall, it remains ineffective overall, indicating that focusing on the most confident frames does not resolve the underlying distribution mismatch.

Overall, the weak performance across both strategies suggests that the learned representations are highly sensitive to dataset-specific artifacts in FaceForensics++, and do not generalize well to the more realistic and diverse manipulations present in Celeb-DF v2.

5.2. True Late Fusion 2D CNN Experiments

The second set of experiments evaluates the true late fusion architecture on FaceForensics++. Unlike the previous approach, multiple frames from the same video are processed jointly and fused before final classification, allowing the model to learn video-level representations directly.

5.2.1. Video-Level Performance on FaceForensics++

The true late fusion model achieves strong performance on the FaceForensics++ test set.

Table 5.4: Video-Level Performance of the Late Fusion on FaceForensics++

Accuracy	Precision	Recall	F1 Score	ROC-AUC
0.9745	0.9167	0.9167	0.9167	0.9943

The model achieves a very high ROC-AUC score, indicating excellent separation between real and fake videos. The balanced precision and recall values further demonstrate stable classification performance across both classes.

Compared to the post-hoc aggregation approach, the true late fusion model achieves competitive overall performance while incorporating cross-frame consistency directly into the inference process.

5.2.2. Classification Report and Confusion Matrix

A more detailed analysis is provided through the classification report and confusion matrix.

Table 5.6: Classification Report for the True Late Fusion Model

Class	Precision	Recall	F1 Score	Support
Real (0)	0.9849	0.9849	0.9849	199
Fake (1)	0.9167	0.9167	0.9167	36

Overall Metric	Score
Accuracy	0.9745
Macro Average F1	0.9508
Weighted Average F1	0.9745

Table 5.7: Confusion Matrix of the True Late Fusion Model

Actual / Predicted	Real	Fake
Real	196	3
Fake	3	33

The confusion matrix shows that the model correctly classifies most of both real and fake videos, with only six total misclassifications.

5.2.3. Training Curves and Visualization results

Figure 5.1: Training Loss Curve

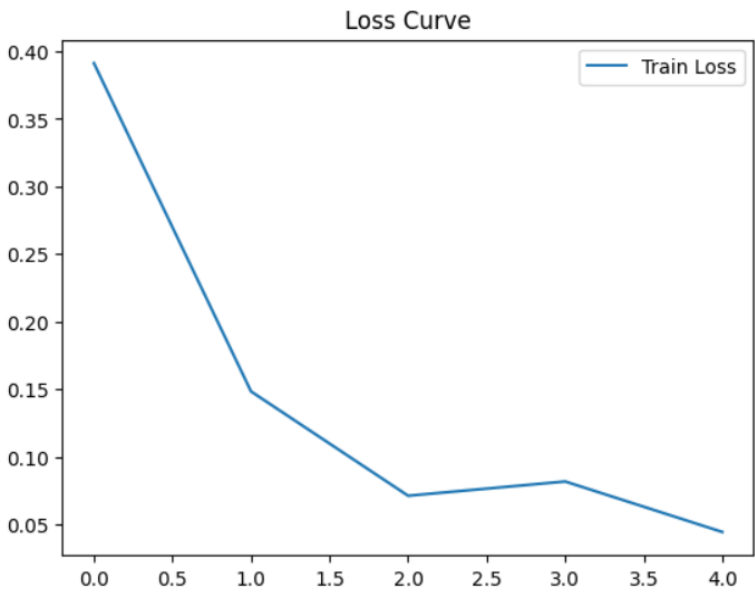


Figure 5.2: Training and Validation Accuracy Curves

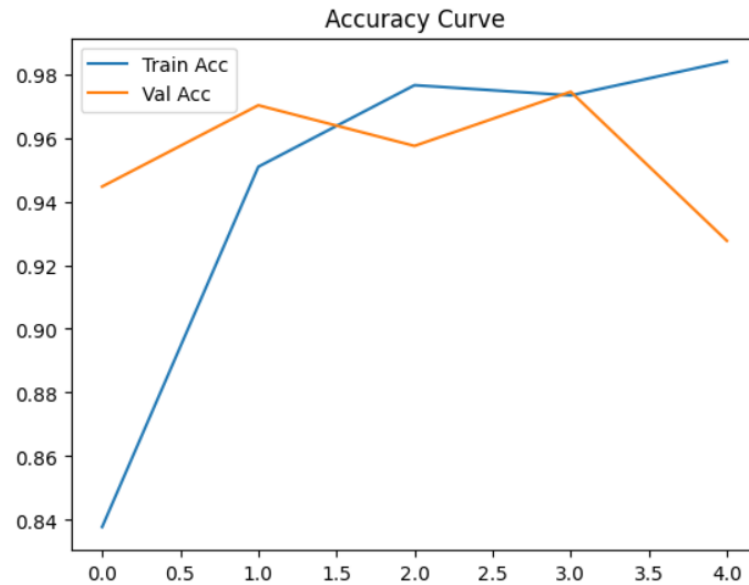
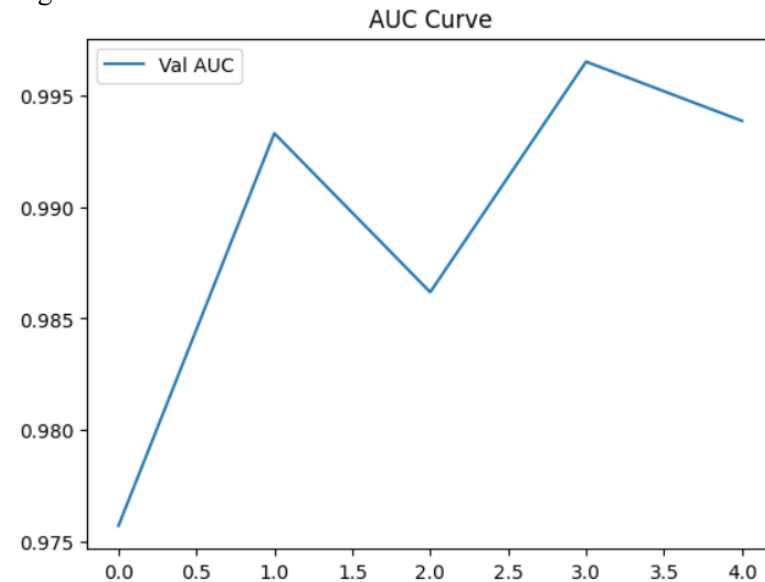


Figure 5.3: Validation ROC-AUC Curve



The training loss curve in Figure 5.1 shows a clear downward trend across epochs, indicating effective optimization and convergence. The loss decreases rapidly in the early stages, dropping from approximately 0.39 to around 0.15, which suggests that the model quickly learns discriminative features from the data. After a minor fluctuation around the middle epochs, the loss continues to decline steadily and stabilizes at a low value near 0.05. Overall, this pattern indicates good convergence behaviour without signs of divergence or unstable training.

The training and validation accuracy curves in Figure 5.2 show a strong learning trend across epochs. Training accuracy steadily increases from about 0.84 to above 0.98, indicating effective feature learning. Validation accuracy also improves early on, reaching around 0.97–0.98, suggesting good generalization.

In later epochs, a slight drop in validation accuracy occurs while training accuracy continues to rise, indicating mild overfitting. However, the gap remains small. Overall, the model shows stable convergence with strong performance and good generalization.

The validation ROC-AUC curve in Figure 5.3 remains consistently high throughout training, ranging from 0.976 at epoch 0 to a peak of ~ 0.997 at epoch 3. The model converges quickly within the first epoch, with a minor dip at epoch 2 before recovering to its best performance. Overall, the model demonstrates strong and stable discrimination ability between real and fake videos across all epochs.

Note: Due to page limitations, the late-fusion inference/testing code, experimental results, and corresponding report are provided in the supplementary material (`celeb_df_late_fusion_inference.ipynb`).

6. Conclusion

The experiments show that both approaches achieve strong performance on FaceForensics++ [3], demonstrating that 2D CNN-based methods are highly effective under in-domain conditions.

Average Probability Aggregation slightly outperforms Majority Voting in the baseline setting, suggesting that preserving probabilistic frame information improves video-level predictions. The true late fusion architecture also achieves highly competitive results by leveraging multi-frame information through feature-level aggregation.

However, the cross-dataset evaluation on Celeb-DF v2 [4] reveals poor generalization performance. Despite strong in-domain results, the model struggles to transfer to unseen manipulation distributions, indicating that the learned representations remain highly dataset-dependent.

Overall, the experiments demonstrate the effectiveness of the proposed approaches for in-domain deepfake detection while also highlighting the continuing challenge of cross-dataset generalization. Future improvements could explore stronger temporal modelling using 3D CNNs or transformer-based architectures, which may better capture motion inconsistencies across frames and improve robustness to unseen manipulation types.

Note: All supplementary materials are provided via a shared Google Drive folder:

<https://drive.google.com/drive/folders/1QrQxNjOjWDA2EpCxnWA-pC5raoTpSF7e?usp=sharing>

REFERENCES

- [1] G. Bertasius, H. Wang, and L. Torresani, “Is Space-Time Attention All You Need for Video Understanding?”, in *Proceedings of the International Conference on Machine Learning (ICML)*, Jul. 2021.
- [2] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid, “ViViT: A Video Vision Transformer,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [3] A. Rössler *et al.*, “FaceForensics++: Learning to Detect Manipulated Facial Images,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [4] Y. Li *et al.*, “Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.